

**ACROPOLIS INSTITUTE OF TECHNOLOGY AND RESEARCH**  
**Department of Information Technology**

*Synopsis*  
on

---

**<< Dynamic Text-to-SQL Generator Using Large Language Models with  
Runtime Schema Awareness >>**

---

**Guided By:**

Prof. Kapil sahu

**Group Members:**

krish Chourasia (0827IT231069)

Manthan jadhav (0827IT231083)

Mansij namdev (0827IT231082)

Lakshya bhagwat (0827IT231079 )

**Link of Project GitHub Repository:**

<https://github.com/KrishChourasia214/text2sql-frontend>

## **1. Introduction**

---

### **1.1. Problem Context (Real-world Scenario)**

In modern organizations, vast amounts of structured data are stored across relational databases that power critical business operations — from customer records and financial transactions to inventory and operational metrics. Accessing and making sense of this data typically requires writing Structured Query Language (SQL) queries, a skill that demands formal training in database management systems, schema comprehension, and query syntax. This technical barrier effectively excludes a large portion of non-technical stakeholders — including business analysts, healthcare professionals, marketing teams, and organizational decision-makers — from directly interacting with data that is essential to their roles.

The challenge is compounded by the diversity and complexity of modern database schemas. A single enterprise application may contain hundreds of tables with complex foreign-key relationships, making it difficult even for experienced developers to construct accurate queries without prior knowledge of the schema. Non-technical users, who may have precise information needs but lack SQL expertise, are forced to rely on dedicated database administrators or technical staff to fetch data on their behalf. This introduces bottlenecks, delays, and miscommunication in data retrieval workflows, ultimately impacting organizational efficiency and decision-making speed.

Furthermore, as organizations increasingly adopt dynamic databases that evolve with business requirements — with tables being added, renamed, or restructured over time — static query interfaces and predefined dashboards quickly become outdated. There is, therefore, a pressing need for an intelligent, adaptive, and accessible interface that enables users of all technical backgrounds to query databases using natural language, without requiring knowledge of underlying database structures.

## 1.2. Motivation & Need

Natural Language Interfaces to Databases (NLIDBs) have long been recognized as a transformative solution to democratize data access. The fundamental idea is straightforward: instead of requiring users to write syntactically precise SQL queries, such interfaces accept questions phrased in plain English and automatically translate them into executable database queries. However, building robust, accurate, and generalizable NLIDBs has historically been a significant research challenge due to the ambiguity of natural language, the diversity of database schemas, and the complexity of SQL semantics.

The emergence of Large Language Models (LLMs) such as GPT-4, LLaMA, and Gemini has fundamentally changed the landscape of natural language processing. These models, trained on vast corpora of text and code, possess remarkable ability to understand the intent behind user queries, reason over structured information, and generate syntactically correct and contextually appropriate SQL statements. When combined with runtime schema awareness — the ability to dynamically introspect and understand the structure of a database at query time — LLMs can produce highly accurate and context-sensitive SQL queries that adapt to any database without the need for manual configuration.

This project is motivated by the recognition that such a system — one that dynamically extracts the database schema at runtime, constructs a rich context for the LLM, and generates validated SQL queries — can significantly lower the barrier to data access, reduce dependency on SQL expertise,

and enhance organizational productivity. By leveraging the power of modern LLMs and building intelligent schema-context pipelines, this project aims to deliver a practical and scalable solution for natural language database querying.

## 2. Literature Survey

---

Research in the field of Text-to-SQL and natural language database interfaces has accelerated significantly with the advent of large-scale pre-trained language models. A comprehensive review of the existing literature reveals both the rapid progress in this domain and the critical limitations that motivate further research.

Early Text-to-SQL systems relied on rule-based approaches and shallow parsing techniques, where manually crafted grammar rules were used to map natural language constructs to SQL patterns. While these systems performed reasonably on narrow domains with simple schemas, they failed to generalize across diverse databases and could not handle complex multi-table queries or ambiguous user intent.

The introduction of deep learning-based methods brought substantial improvements. Seq2Seq architectures with attention mechanisms, such as those proposed by Dong and Lapata (2016), demonstrated the feasibility of learning SQL generation as a sequence-to-sequence translation task. However, these models required large amounts of annotated training data and struggled with unseen database schemas.

The Spider benchmark (Yu et al., 2018) established a gold standard for cross-domain Text-to-SQL evaluation, introducing complex queries across 200 databases. Subsequent models like IRNet, RATSQL, and BRIDGE proposed increasingly sophisticated schema-linking mechanisms that aligned natural language entities with database columns and tables.

With the rise of transformer-based pre-trained language models, systems like PICARD (Scholak et al., 2021) introduced constrained decoding strategies that enforced SQL grammar during beam search, significantly improving syntactic correctness. More recent work has explored the use of instruction-tuned LLMs such as GPT-4 and Code Llama in few-shot and zero-shot settings, where the model is provided with schema information in the prompt and asked to generate SQL directly.

Research on Retrieval-Augmented Generation (RAG) for SQL (Guo et al., 2023) has demonstrated that augmenting LLM prompts with relevant schema fragments, sample data, and few-shot examples from similar queries improves accuracy on complex databases. This approach aligns closely with the runtime schema awareness methodology adopted in this project.

Studies on query optimization within LLM-generated SQL (Liu et al., 2024) have shown that postgeneration validation and correction strategies can catch and repair common errors such as incorrect table joins, wrong aggregation functions, and invalid column references, thereby improving the reliability of generated queries.

Despite these advances, existing Text-to-SQL systems exhibit several notable limitations:

- Most systems require a pre-defined, static schema provided at training or configuration time and cannot adapt to databases that change at runtime.
- Many approaches perform poorly on schemas with ambiguous or abbreviated column names that require semantic understanding.
- Hallucination of non-existent table names or column names remains a persistent challenge in LLM-based generation.
- Few systems provide end-to-end validation pipelines that verify the syntactic and semantic correctness of generated queries before execution.
- Integration with production database systems (MySQL, PostgreSQL) in a lightweight, deployable web application remains underexplored.

The key technology areas relevant to this project include:

| Technology Area                      | Relevance to This Project                        |
|--------------------------------------|--------------------------------------------------|
| Text-to-SQL Systems                  | Core task: converting natural language to SQL    |
| Large Language Models (LLMs)         | Central engine for query generation              |
| Natural Language Processing (NLP)    | Intent extraction and query understanding        |
| Database Schema Understanding        | Schema extraction, parsing, and context building |
| Retrieval-Augmented Generation (RAG) | Schema-augmented prompting for LLMs              |
| Runtime Schema Awareness             | Dynamic schema introspection at query time       |
| Query Optimization & Validation      | Post-generation SQL correctness checks           |

## 2.1. Problem Statement

*To design and develop a system that converts natural language queries into accurate and executable SQL statements using Large Language Models and runtime schema awareness, enabling users — regardless of their SQL expertise — to retrieve structured information from relational databases through a simple, intuitive web interface.*

## 3. Proposed Solution Overview

---

The proposed system is a web-based application that provides an intelligent natural language interface for querying relational databases. The system is designed around the principle of runtime schema awareness, wherein the database schema is dynamically extracted at query time rather than being hardcoded or pre-loaded, enabling the system to work accurately with any database without manual reconfiguration.

The workflow of the proposed system is as follows:

- The user accesses the web interface and enters a natural language question describing the data they wish to retrieve (e.g., "Show me the total sales by region for the last quarter").
- The backend module dynamically connects to the target database and extracts the complete schema at runtime — including table names, column names, data types, primary keys, and foreign key relationships.
- A Schema Context Builder processes the extracted schema and identifies the tables and relationships most relevant to the user's query using semantic similarity and keyword matching.
- The relevant schema context is formatted into a structured prompt, which is submitted to the Large Language Model along with the original natural language query and few-shot examples.
- The LLM analyzes the prompt and generates a syntactically correct SQL query tailored to the specific schema context provided.
- The generated SQL is passed through a Query Validator that checks for syntactic correctness, references to valid table and column names, and logical consistency with the schema.
- If the query passes validation, it is executed against the database engine, and the results are returned to the user in a structured, readable format through the web interface.
- If validation fails, the system logs the error and optionally re-prompts the LLM with the error message for self-correction.

Runtime schema awareness is a critical differentiator of this system. By extracting the schema fresh at each query session, the system remains accurate even as the database structure evolves — new tables are automatically discovered, renamed columns are correctly identified, and obsolete structures are no longer referenced. This significantly reduces the risk of hallucination, where LLMs may generate queries referencing table or column names that do not exist in the actual database. By grounding the LLM with real, live schema information, the system ensures that generated queries are consistently accurate and executable.

## 4. Objectives

---

The main objectives of the proposed system are:

- To develop a web-based platform that accepts natural language queries from users and automatically translates them into accurate SQL statements using Large Language Models.
- To implement a runtime schema extraction module that dynamically connects to and introspects any relational database, retrieving table structures, column metadata, and relationships at query time.
- To build a Schema Context Builder that intelligently selects the most relevant subset of the database schema for each user query, reducing prompt size and improving LLM focus.
- To integrate state-of-the-art Large Language Models (OpenAI GPT-4, LLaMA, or Gemini) with a structured prompting strategy that incorporates schema context and few-shot examples to maximize query generation accuracy.
- To implement a Query Validation module that verifies the syntactic and semantic correctness of LLM-generated SQL queries before execution, catching and correcting common errors.
- To reduce dependency on SQL expertise among end users, enabling non-technical stakeholders — including business analysts, healthcare workers, and domain experts — to independently access and query structured data.
- To ensure compatibility with multiple popular relational database management systems, including MySQL and PostgreSQL, through a database-agnostic architecture.
- To provide a clean, intuitive web interface that presents query results in a structured, readable format suitable for non-technical users.

## 5. Contributions

---

- Seamless integration of Large Language Models with production relational database systems, enabling end-to-end natural language query processing within a unified web application.

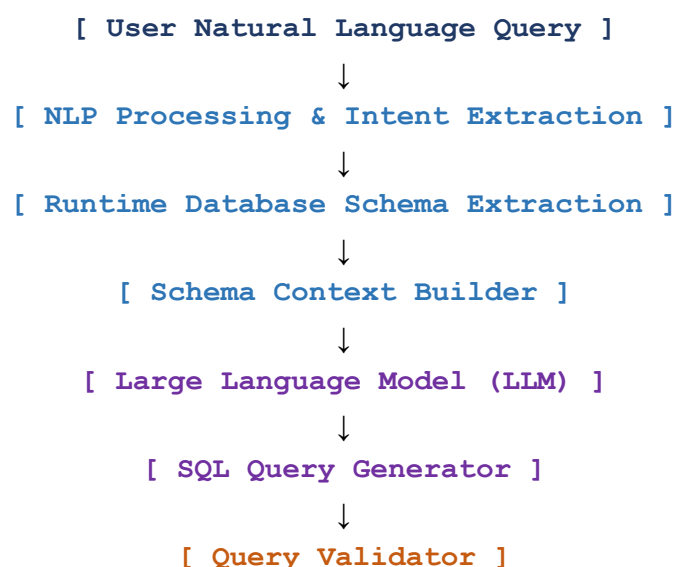
- Introduction of a runtime schema-aware query generation pipeline that dynamically extracts and leverages live database schema information at query time, eliminating the need for static schema pre-configuration.
- Development of a Schema Context Builder that intelligently filters and structures schema metadata into LLM-consumable prompts, improving query accuracy and reducing token consumption.
- Implementation of a multi-stage Query Validation module that intercepts and corrects syntactic and semantic errors in LLM-generated SQL before database execution, significantly improving reliability.
- Dynamic adaptation to evolving databases — the system automatically accommodates schema changes without requiring reconfiguration, ensuring long-term usability in production environments.
- A significant reduction in barriers to data access, empowering non-technical users to independently retrieve database insights and reducing organizational dependency on dedicated database administrators.

## 6. Theoretical Analysis

---

### 6.1 Block Diagram

The diagram "FIG: 01" represents the complete end-to-end workflow of the Dynamic Text-to-SQL Generator system. The pipeline begins with user input at the web interface and proceeds through NLP processing, runtime schema extraction, LLM-based SQL generation, validation, and database execution, culminating in the display of query results.



↓  
[ Database Engine (MySQL / PostgreSQL) ]  
↓  
[ Result Display to User ]

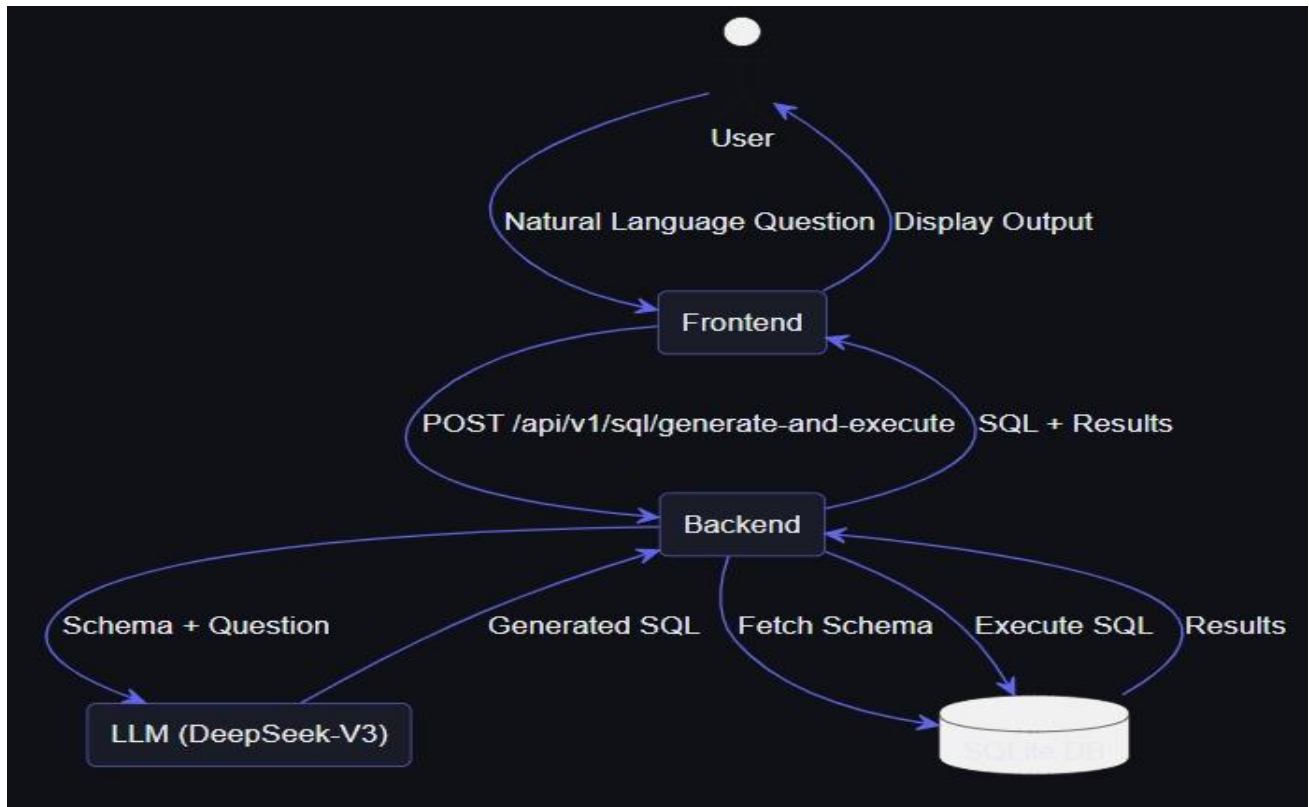


FIG: 01 End-to-end workflow of the Dynamic Text-to-SQL Generator

Each block in the diagram serves a specific purpose within the pipeline:

- **User Natural Language Query:** The user submits a free-form natural language question through the web interface, such as "List all customers who placed more than three orders in January 2024."
- **NLP Processing & Intent Extraction:** The user input is preprocessed to tokenize, normalize, and extract semantic intent, which guides schema element selection in subsequent steps.
- **Runtime Database Schema Extraction:** The backend dynamically connects to the configured database and introspects its schema using SQLAlchemy's inspection API, retrieving all table names, column names, data types, and relationships.
- **Schema Context Builder:** Using semantic similarity between the user query and schema elements, this module constructs a concise, structured schema context (formatted as table definitions) to be included in the LLM prompt.



- **Large Language Model (LLM):** The LLM receives a structured prompt containing the user query, schema context, and few-shot examples, and generates a candidate SQL query.
- **SQL Query Generator:** This layer formats and refines the LLM's output into an executable SQL statement, handling edge cases such as dialect-specific syntax.
- **Query Validator:** The generated SQL is parsed and validated against the live schema to detect invalid table names, column references, syntax errors, or logical inconsistencies.
- **Database Engine:** The validated query is executed against the target relational database (MySQL or PostgreSQL), and the results are retrieved.
- **Result Display:** The query results are rendered in the web interface in a tabular format, with optional natural language summarization.

## 6.2 Hardware/Software Designing

### 1. Hardware Requirements

| Component             | Requirement                                  |
|-----------------------|----------------------------------------------|
| CPU                   | Intel Core i5 / AMD Ryzen 5 or equivalent    |
| RAM                   | 8 GB (16 GB recommended for LLM inference)   |
| Storage               | 20 GB free disk space                        |
| Internet Connectivity | Required for LLM API calls (OpenAI / Gemini) |
| Display               | Standard monitor, 1080p resolution           |

### 2. Software Requirements

| Layer / Component       | Technology Stack                               |
|-------------------------|------------------------------------------------|
| Operating System        | Windows 10/11, Linux (Ubuntu 20.04+), or macOS |
| Frontend                | React.js (with Axios for API communication)    |
| Backend Framework       | Python & FastAPI                               |
| LLM Integration         | OpenAI GPT-4 / Meta LLaMA / Google Gemini API  |
| Database                | MySQL / PostgreSQL                             |
| ORM & Schema Inspection | SQLAlchemy                                     |

|                   |                                             |
|-------------------|---------------------------------------------|
| Development Tools | VS Code, GitHub, Postman                    |
| Deployment        | Docker (containerized), Uvicorn ASGI server |

## 7. Applications

---

The proposed Dynamic Text-to-SQL Generator system has broad applicability across a wide range of domains and use cases:

- **Business Intelligence & Analytics:** Business analysts and executives can query enterprise databases directly using natural language to generate on-demand reports, reducing dependence on data engineering teams and accelerating insight generation.
- **Database Administration & Exploration:** Database administrators and developers can use the system as a rapid exploration tool for unfamiliar schemas, quickly retrieving data from complex databases without manually constructing intricate JOIN queries.
- **Educational Institutions:** Universities and training institutes can deploy the system as a learning aid, allowing students to explore relational databases interactively and observe the SQL equivalents of their natural language questions.
- **Healthcare Information Systems:** Clinicians, researchers, and hospital administrators can query patient databases, treatment records, and clinical trial data in plain English, enabling timely data-driven decisions without requiring SQL knowledge.
- **E-Commerce & Retail Analytics:** Product managers and marketing teams can independently query sales, inventory, and customer behavior databases to track performance metrics and identify trends without technical assistance.
- **Banking & Financial Services:** Financial analysts can retrieve transactional data, customer records, and risk metrics using natural language queries, improving the speed and accessibility of financial reporting.
- **Government & Public Sector Data Portals:** Government agencies can expose structured public data repositories through natural language interfaces, enabling citizens, researchers, and policymakers to access information without technical expertise.
- **Enterprise Reporting & Internal Tools:** Organizations can integrate the system into internal knowledge management and reporting workflows, enabling employees across departments to self-serve data requests independently.

## References

---

1. Yu, T., Zhang, R., Yang, K., Yasunaga, M., Wang, D., Li, Z., & Radev, D. (2018). Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. Proceedings of EMNLP 2018. <https://arxiv.org/abs/1809.08887>
2. Scholak, T., Schucher, N., & Bahdanau, D. (2021). PICARD: Parsing Incrementally for Constrained AutoRegressive Decoding from Language Models. Proceedings of EMNLP 2021. <https://arxiv.org/abs/2109.05093>
3. Guo, J., Zhan, Z., Gao, Y., Xiao, Y., Lou, J. G., Liu, T., & Zhang, D. (2023). Towards Complex Text-to-SQL in Cross-Domain Database with Intermediate Representation. ACL 2023. <https://arxiv.org/abs/1905.08205>
4. Pourreza, M., & Rafiei, D. (2024). DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with SelfCorrection. NeurIPS 2023. <https://arxiv.org/abs/2304.11015>
5. Sun, Y., et al. (2023). SQL-PaLM: Improved Large Language Model Adaptation for Text-to-SQL. arXiv preprint. <https://arxiv.org/abs/2306.00739>
6. Lewis, P., et al. (2023). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. Advances in Neural Information Processing Systems. <https://arxiv.org/abs/2005.11401>
7. OpenAI. (2024). GPT-4 Technical Report. <https://openai.com/research/gpt-4>
8. Touvron, H., et al. (2023). LLaMA 2: Open Foundation and Fine-Tuned Chat Models. Meta AI. <https://arxiv.org/abs/2307.09288>
9. Deng, N., Chen, Y., & Zhang, Y. (2023). Recent Advances in Text-to-SQL: A Survey of What We Have and What We Expect. arXiv preprint. <https://arxiv.org/abs/2208.10099>
10. SQLAlchemy Documentation. (2024). Runtime Schema Inspection and ORM Integration. <https://docs.sqlalchemy.org/en/20/core/reflection.html>
11. FastAPI Documentation. (2024). Building High-Performance APIs with Python. <https://fastapi.tiangolo.com/>
12. Liu, X., et al. (2024). A Comprehensive Evaluation of LLM-Based Text-to-SQL Systems on Real-World Databases. SIGMOD 2024.